C Code for Dijkstra's Algorithm

Below is the C code implementing Dijkstra's algorithm for finding the shortest path:

```c
#include <stdio.h>
#include <limits.h>

#define V 17 // Number of vertices

// Function to find the vertex with the minimum distance value
int minDistance(int dist[], int sptSet[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (sptSet[v] == 0 && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to print the shortest path from source to a target
void printPath(int parent[], int target) {
    if (parent[target] == -1) {
        printf("%d ", target + 1); // Convert to 1-indexed for readability
        return;
    }
    printPath(parent, parent[target]);
    printf("%d ", target + 1);
}

// Function to implement Dijkstra's algorithm
void dijkstra(int graph[V][V], int src, int target) {
    int dist[V];        // Array to store the shortest distance from the source to each vertex
    int sptSet[V];      // Shortest path tree set (true if vertex is included in the shortest path)
    int parent[V];      // Array to store the shortest path tree

    // Initialize distances as infinite and sptSet[] as false
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        sptSet[i] = 0;
```

```c
        parent[i] = -1; // No parent initially
    }

    // Distance from source to itself is always 0
    dist[src] = 0;

    // Find the shortest path for all vertices
    for (int count = 0; count < V - 1; count++) {
        // Pick the minimum distance vertex from the set of vertices not yet processed
        int u = minDistance(dist, sptSet);

        // Mark the picked vertex as processed
        sptSet[u] = 1;

        // Update the distance value of the adjacent vertices of the picked vertex
        for (int v = 0; v < V; v++) {
            // Update dist[v] if not in sptSet, there is an edge, and total distance is smaller
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u; // Record the path
            }
        }
    }

    // Output the shortest path and distance
    printf("Shortest distance from %d to %d: %d\n", src + 1, target + 1, dist[target]);
    printf("Path: ");
    printPath(parent, target);
    printf("\n");
}

int main() {
    int graph[V][V] = {
        {0, 750, 650, 700, 400, 375, 500, 510, 500, 510, 350, 125, 350, 450, 475, 450, 600},
        {750, 0, 100, 300, 650, 650, 750, 760, 750, 760, 600, 625, 400, 700, 705, 700, 800},
        {650, 100, 0, 400, 600, 610, 550, 560, 550, 560, 450, 500, 550, 510, 515, 510, 650},
        {700, 300, 400, 0, 350, 360, 250, 260, 250, 260, 375, 425, 450, 425, 430, 425, 650},
        {400, 650, 600, 350, 0, 10, 150, 155, 150, 155, 200, 225, 300, 225, 230, 225, 300},
```

```c
        {375, 650, 610, 360, 10, 0, 150, 155, 150, 155, 200, 225, 300, 225, 230, 225, 300},
        {500, 750, 550, 250, 150, 150, 0, 5, 5, 5, 225, 310, 350, 250, 255, 250, 100},
        {510, 760, 560, 260, 155, 155, 5, 0, 5, 5, 225, 310, 350, 250, 225, 250, 100},
        {500, 750, 550, 250, 150, 150, 5, 5, 0, 5, 225, 310, 350, 250, 225, 250, 100},
        {510, 760, 560, 260, 155, 155, 5, 5, 5, 0, 225, 310, 350, 250, 225, 250, 100},
        {350, 600, 450, 375, 200, 200, 225, 225, 225, 225, 0, 175, 125, 50, 50, 50, 150},
        {125, 625, 500, 425, 225, 225, 310, 310, 310, 310, 175, 0, 100, 150, 100, 150, 275},
        {350, 400, 550, 450, 300, 300, 350, 350, 350, 350, 125, 100, 0, 125, 100, 125, 225},
        {450, 700, 510, 425, 225, 225, 250, 250, 250, 250, 50, 150, 125, 0, 15, 15, 100},
        {475, 705, 515, 430, 230, 230, 255, 225, 225, 225, 50, 100, 100, 15, 0, 15, 125},
        {450, 700, 510, 425, 225, 225, 250, 250, 250, 250, 50, 150, 125, 15, 15, 0, 105},
        {600, 800, 650, 650, 300, 300, 100, 100, 100, 100, 150, 275, 225, 100, 125, 105, 0}
    };

    int source, target;

    // User input for source and target vertices
    printf("Enter the source vertex (1 to 17): ");
    scanf("%d", &source);
    printf("Enter the target vertex (1 to 17): ");
    scanf("%d", &target);

    // Validate input
    if (source < 1 || source > V || target < 1 || target > V) {
        printf("Invalid vertex.\n");
        return 1;
    }

    // Call Dijkstra's algorithm
    dijkstra(graph, source - 1, target - 1); // Adjust 1-indexed input to 0-indexed

    return 0;
}
```

Below are the sample results of running the code:

```
Enter the source vertex (1 to 17): 4
Enter the target vertex (1 to 17): 7
Shortest distance from 4 to 7: 250
Path: 4 7
```

Figure 1: The shortest path from Library to Phase 2 hostel.

```
Enter the source vertex (1 to 17): 14
Enter the target vertex (1 to 17): 2
Shortest distance from 14 to 2: 515
Path: 14 15 13 2
```

Figure 2: The shortest path from Engineering Block to Cricket Ground.

Instructions for Running the Code

Prerequisites

Ensure the following tools are installed on your system:

- A C compiler (e.g., GCC).
- A terminal or command prompt to compile and execute the program.

Steps to Run the Code

1. Save the source code into a file named `dijkstra.c`.
2. Open a terminal or command prompt and navigate to the directory where `dijkstra.c` is located.
3. Compile the program using the GCC compiler. Run the following command:

   ```
   gcc dijkstra.c -o dijkstra
   ```

   This command compiles the source code and generates an executable file named `dijkstra`.
4. Run the program using the command:

   ```
   ./dijkstra
   ```

5. The program will prompt you to enter the source and target vertices. Enter these as integers (e.g., 1, 2, ..., 17).
6. View the output, which will display the shortest distance and the path between the specified vertices.

Notes

- Ensure the input values for vertices are within the valid range (1 to 17 in this case).
- If the program doesn't run as expected, check for errors in the source code or ensure GCC is installed and configured correctly.
- If you're using an operating system other than Linux or macOS, adjust the commands accordingly (e.g., for Windows, replace `./dijkstra` with `dijkstra.exe`).